Python Exception Handling -

1. Introduction / परिचय

What is Exception? / Exception क्या है?

English: An exception is an error that occurs during program execution, disrupting the normal flow of the program.

Hindi: Exception एक error है जो program execution के दौरान होती है और program के normal flow को बाधित करती है।

Why Exception Handling? / Exception Handling क्यों?

- Prevents program crashes / Program को crash होने से बचाता है
- Provides error information / Error की जानकारी देता है
- Allows graceful error recovery / Error से gracefully recover करने देता है

---

2. Basic Syntax / बुनियादी Syntax

Try-Except Block

```
# English Example
try:
    # Code that might raise an exception
    result = 10 / 0
except:
    # Code to handle the exception
    print("An error occurred")
```

Output:

```
An error occurred (एक error आई है)
```

---

3. Specific Exception Handling / विशिष्ट Exception Handling

Catching Specific Exceptions

```
# English Example
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(f"Result: {result}")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid input! Please enter a number.")
```

---

4. Common Built-in Exceptions / सामान्य Built-in Exceptions

Exception	Description (English)	विवरण (Hindi)
`ZeroDivisionError`	Division by zero	शून्य से विभाजन
`ValueError`	Invalid value	गलत मान
`TypeError`	Wrong data type	गलत data type
`IndexError`	Index out of range	Index range से बाहर
`KeyError`	Key not found in dictionary	Dictionary में key नहीं मिली
`FileNotFoundError`	File doesn't exist	File मौजूद नहीं
`AttributeError`	Invalid attribute	गलत attribute
`ImportError`	Module import failed	Module import fail हुआ
`NameError`	Variable not defined	Variable define नहीं है
`IOError`	Input/Output error	Input/Output error

---

5. Multiple Exception Handling / एकाधिक Exception Handling

Method 1: Separate Except Blocks

```
# English Example
try:
    file = open("data.txt", "r")
    content = file.read()
    number = int(content)
    result = 100 / number
except FileNotFoundError:
    print("File not found!")
except ValueError:
    print("File contains invalid data!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
finally:
    print("Execution completed")
```

Method 2: Tuple of Exceptions

```
try:
    # Some code
    pass
except (TypeError, ValueError, KeyError) as e:
    print(f"Error occurred: {e}")
```

---

6. Else Clause / Else खंड

English: The `else` block executes if no exception occurs.

Hindi: `else` block तब execute होता है जब कोई exception नहीं आती।

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid input!")
else:
    print(f"Success! Result is {result}")
```

7. Finally Clause / Finally खंड

English: The `finally` block always executes, whether an exception occurs or not. Used for cleanup operations.

Hindi: `finally` block हमेशा execute होता है, चाहे exception आए या न आए। Cleanup operations के लिए use होता है।

```
# English Example
try:
    file = open("data.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("File not found!")
finally:
    print("Closing file...")
    try:
        file.close()
    except:
        pass
```

8. Raising Exceptions / Exception Raise करना

Using `raise` Keyword

```
def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    elif age < 18:
        raise Exception("Must be 18 or older")
    else:
        print("Access granted")

try:
    check_age(15)
except ValueError as e:
    print(f"Value Error: {e}")
except Exception as e:
    print(f"Error: {e}")
```

---

9. Custom Exceptions / अपनी Exception बनाना

```
# English Example
class InsufficientBalanceError(Exception):
    """Custom exception for insufficient balance"""
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        self.message = f"Insufficient balance: {balance}, tried to
withdraw: {amount}"
        super().__init__(self.message)

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError(balance, amount)
    return balance - amount

try:
    new_balance = withdraw(1000, 1500)
except InsufficientBalanceError as e:
    print(e.message)

# Hindi Example
class InsufficientBalanceError(Exception):
    """अपर्याप्त balance के लिए custom exception"""
    def __init__(self, balance, amount):
```

```
        self.balance = balance
        self.amount = amount
        self.message = f"अपर्याप्त balance: {balance}, निकालने की कोशिश: 
{amount}"
        super().__init__(self.message)

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError(balance, amount)
    return balance - amount

try:
    new_balance = withdraw(1000, 1500)
except InsufficientBalanceError as e:
    print(e.message)
```

---

10. Exception Chaining

```
try:
    try:
        result = 10 / 0
    except ZeroDivisionError as e:
        raise ValueError("Invalid calculation") from e
except ValueError as e:
    print(f"Error: {e}")
    print(f"Original cause: {e.__cause__}")
```

---

11. Assert Statement / Assert Statement

English: Used for debugging. Raises `AssertionError` if condition is False.

Hindi: Debugging के लिए use होता है। अगर condition False है तो `AssertionError` raise करता है।

```
# English Example
def calculate_average(numbers):
    assert len(numbers) > 0, "List cannot be empty"
    return sum(numbers) / len(numbers)

try:
    avg = calculate_average([])
except AssertionError as e:
    print(f"Assertion Error: {e}")
```

12. Best Practices / सर्वोत्तम Practices

English:

1. **Catch specific exceptions** - Don't use bare `except:`
2. **Keep try blocks small** - Only include code that might raise exceptions
3. **Use finally for cleanup** - Close files, connections, etc.
4. **Don't suppress exceptions** - Log them or handle appropriately
5. **Create custom exceptions** - For application-specific errors
6. **Use exception chaining** - Preserve error context
7. **Avoid using exceptions for flow control** - Use conditionals instead

Hindi:

1. **Specific exceptions catch करें** - Bare `except:` use न करें
2. **Try blocks छोटे रखें** - केवल वही code जो exception raise कर सकता है
3. **Cleanup के लिए finally use करें** - Files, connections आदि बंद करें
4. **Exceptions को suppress न करें** - उन्हें log या handle करें
5. **Custom exceptions बनाएं** - Application-specific errors के लिए
6. **Exception chaining use करें** - Error context preserve करें
7. **Flow control के लिए exceptions avoid करें** - Conditionals use करें

---

13. Practical Examples / व्यावहारिक उदाहरण

Example 1: File Operations

```
# English
def read_file_safely(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
            return content
    except FileNotFoundError:
        print(f"Error: {filename} not found")
        return None
    except PermissionError:
        print(f"Error: No permission to read {filename}")
        return None
    except Exception as e:
        print(f"Unexpected error: {e}")
        return None
```

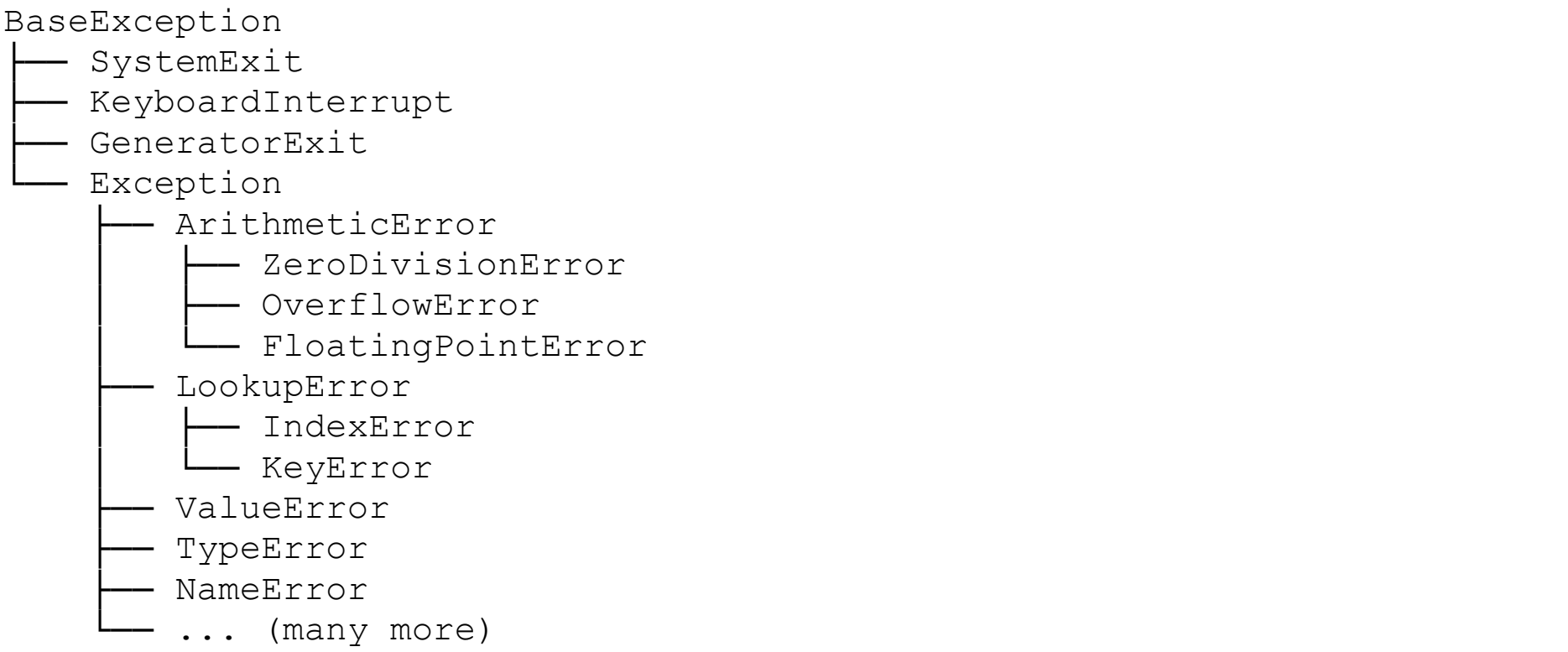
Example 2: User Input Validation

```
# English
def get_valid_number():
    while True:
        try:
            number = int(input("Enter a number: "))
            if number < 0:
                raise ValueError("Number must be positive")
            return number
        except ValueError as e:
            print(f"Invalid input: {e}")
            print("Please try again.")
```

Example 3: Database Operations

```
# English
def connect_database(db_name):
    connection = None
    try:
        # Simulated database connection
        connection = f"Connected to {db_name}"
        print(connection)
        # Perform operations
        return True
    except ConnectionError:
        print("Failed to connect to database")
        return False
    except Exception as e:
        print(f"Database error: {e}")
        return False
    finally:
        if connection:
            print("Closing database connection")
```

14. Exception Hierarchy / Exception पदानुक्रम



---

15. Summary / सारांश

English:

- Exception handling prevents program crashes
- Use try-except blocks to catch and handle errors
- Always catch specific exceptions when possible
- Use finally for cleanup operations
- Create custom exceptions for application-specific errors
- Follow best practices for robust code

Hindi:

- Exception handling program को crash होने से बचाता है
- Errors को catch और handle करने के लिए try-except blocks use करें
- जब संभव हो specific exceptions catch करें
- Cleanup operations के लिए finally use करें
- Application-specific errors के लिए custom exceptions बनाएं
- Robust code के लिए best practices follow करें

Quick Reference Card / त्वरित संदर्भ

```
# Basic Structure / मूल संरचना
try:
    # risky code / जोखिम वाला code
    pass
except SpecificError:
    # handle specific error / specific error handle करें
    pass
except Exception as e:
    # handle any other error / कोई अन्य error handle करें
    pass
else:
    # runs if no exception / exception न होने पर चलता है
    pass
finally:
    # always runs / हमेशा चलता है
    pass
```